

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ «Информатика и системы управления»

КАФЕДРА ИУ7 «Программное обеспечение ЭВМ и информационные технологии»

КУРСОВАЯ РАБОТА

НА ТЕМУ:

«Драйвер для геймпада Xbox Series S»

Студент

_____ **А. М. Трохан**
(Подпись, дата) (И.О.Фамилия)

Руководитель

_____ **Н. Ю. Рязанова**
(Подпись, дата) (И.О.Фамилия)

2025 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ
Заведующий кафедрой ИУ7
(Индекс)
И.В. Рудаков
(И.О.Фамилия)
« » 20 г.

З А Д А Н И Е
на выполнение курсового проекта

по дисциплине Операционные системы

Студент группы ИУ7-72Б

Трохан Андрей Максимович
(Фамилия, имя, отчество)

Тема курсового проекта Драйвер для геймпада Xbox Series S.

Направленность КП (учебный, исследовательский, практический, производственный, др.)
учебный

Источник тематики (кафедра, предприятие, НИР) кафедра

График выполнения проекта: 25% к 4 нед., 50% к 7 нед., 75% к 11 нед., 100% к 14 нед.

Задание Разработать для геймпада Xbox Series S, выполняющий функции мыши. Левый стик выполняет функцию горизонтального движения курсора, правый стик — вертикального. Функцию левой кнопки мыши выполняет кнопка A. Функцию правой кнопки мыши выполняет крестовина.

Оформление курсового проекта:

Расчетно-пояснительная записка на 25-30 листах формата А4.

Перечень графического (иллюстративного) материала (чертежи, плакаты, слайды и т.п.)

На защиту проекта должна быть представлена презентация. На слайдах должны быть: постановка задачи, использованные методы и алгоритмы, расчётные соотношения, структура комплекса программ, диаграмма классов, интерфейс, характеристики ПО, результаты исследования.

Дата выдачи задания « » 20 г.

Руководитель курсового проекта

Н. Ю. Рязанова
(Подпись, дата) (И.О.Фамилия)

Студент

А. М. Трохан
(Подпись, дата) (И.О.Фамилия)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Аналитический раздел	5
1.1 Постановка задачи	5
1.2 Драйверы в Linux	5
1.3 USB-подсистема ядра Linux	6
1.3.1 Структура USB-устройства	6
1.3.2 Структура USB-драйвера	6
1.3.3 Структура URB	7
1.4 Подсистема ввода Linux	8
1.4.1 Структура устройства ввода	8
1.4.2 Типы событий ввода	9
1.5 Протокол взаимодействия с Xbox Series S	10
1.5.1 Структура входного пакета	10
1.6 Выводы	11
2 Конструкторский раздел	13
2.1 Последовательность действий ПО	13
2.2 Архитектура драйвера	13
2.3 Алгоритм обработки ввода	14
2.4 Алгоритм эмуляции мыши	14
2.5 Инициализация контроллера	15
3 Технологический раздел	16
3.1 Выбор языка и среды программирования	16
3.2 Функциональность разработанного ПО	16
3.3 Структура данных драйвера	17
3.4 Инициализация USB-драйвера	17
3.5 Параметры модуля	18
3.6 Листинг загружаемого модуля ядра	18
4 Исследовательский раздел	19
4.1 Загрузка модуля	19

4.2	Тестирование кнопок мыши	19
4.3	Тестирование движения курсора	19
	ЗАКЛЮЧЕНИЕ	21
	ПРИЛОЖЕНИЕ А	22
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	22
	ПРИЛОЖЕНИЕ Б	37

ВВЕДЕНИЕ

Возможность гибко задавать часто используемые действия системы напрямую влияет на удобство взаимодействия пользователя с компьютером. Современные геймпады, такие как Xbox Series S, предоставляют широкие возможности для альтернативного способа управления компьютером, включая функции мыши. Это особенно актуально в ситуациях, когда использование традиционной мыши затруднено или невозможно, например при подключении компьютера к телевизору, в домашних кинотеатрах или при работе с медиацентрами.

Геймпад Xbox Series S, благодаря своим точным аналоговым стикам и эргономичной конструкции, идеально подходит для реализации функций мыши. Разработка драйвера, позволяющего использовать данный контроллер в качестве полноценного устройства ввода типа "мышь" является актуальной задачей, которая расширяет функциональность контроллера и повышает удобство работы пользователя в различных сценариях.

Данная работа посвящена разработке драйвера для геймпада Xbox Series S, который преобразует контроллер в устройство мыши с поддержкой перемещения курсора, нажатия кнопок и прокрутки.

1 Аналитический раздел

1.1 Постановка задачи

В соответствии с заданием на курсовую работу необходимо разработать драйвер для геймпада Xbox Series S, выполняющий функции мыши с возможностью регулировки чувствительности.

Для выполнения задания требуется решить следующие задачи:

1. провести анализ существующих подходов к обработке устройств ввода в Linux;
2. исследовать протоколы взаимодействия с контроллером Xbox Series S;
3. разработать алгоритмы преобразования ввода геймпада в события мыши;
4. реализовать драйвер в виде загружаемого модуля ядра Linux;
5. провести тестирование и исследование разработанного ПО.

1.2 Драйверы в Linux

В современных ядрах Linux изменение функциональности внешних устройств возможно с помощью драйверов, представляющих собой загружаемые модули ядра. Драйверы полностью скрывают детали, касающиеся работы устройства и предоставляют чёткий программный интерфейс для работы с аппаратурой.

Для реализации драйвера Xbox Series S выбран подход загружаемого модуля ядра, который обеспечивает:

- Гибкость конфигурации без перезагрузки системы;
- Автоматическое определение подключения контроллера;
- Интеграцию с существующей подсистемой ввода Linux;
- Возможность параметрической настройки через модульные параметры.


```

6
7  void (*disconnect)(struct usb_interface *intf); // Функция отк
    лючения
8
9  int (*suspend)(struct usb_interface *intf, // Приостановка
10                pm_message_t message);
11
12  int (*resume)(struct usb_interface *intf); // Возобновление
13
14  const struct usb_device_id *id_table; // Таблица ID устройс
    тв
15  struct module *owner; // Владелец модуля
16 };

```

1.3.3 Структура URB

Для асинхронного обмена данными с USB-устройством используется URB (USB Request Block):

Листинг 1.3 – Структура urb

```

1 struct urb {
2     struct usb_device *dev; // USB-устройство
3     unsigned int pipe; // Канал передачи
4
5     void *transfer_buffer; // Буфер данных
6     u32 transfer_buffer_length; // Размер буфера
7     dma_addr_t transfer_dma; // DMA-адрес буфера
8
9     int status; // Статус операции
10    u32 actual_length; // Фактическая длина
11
12    void *context; // Контекст для
        callback
13    usb_complete_t complete; // Функция завершения
14
15    unsigned int transfer_flags; // Флаги передачи
16    unsigned int interval; // Интервал для
        interrupt
17 };

```

Для драйвера Xbox Series S используется тип передачи данных **interrupt**, который предназначен для коротких пакетов со спонтанным характером (до

64 байт на полной скорости). Это оптимальный выбор для устройств ввода, передающих координаты и состояние кнопок.

Каждый USB-драйвер должен определить:

- Таблицу идентификаторов устройств (`struct usb_device_id`), связывающую драйвер с конкретными vendor (0x045e для Microsoft) и product ID (0x0b12 для Xbox Series S);
- Структуру драйвера (`usb_driver`) с callback-функциями (`probe`, `disconnect`, `id_table`);
- Функцию инициализации модуля через макрос `module_usb_driver`.

1.4 Подсистема ввода Linux

Для интеграции с системой ввода Linux используется структура `struct input_dev`, которая является фундаментальным компонентом подсистемы ввода/вывода. Данная подсистема абстрагирует физические устройства ввода в единую модель, что позволяет приложениям получать события от любых устройств через унифицированный интерфейс.

1.4.1 Структура устройства ввода

Листинг 1.4 – Структура `input_dev`

```
1 struct input_dev {  
2     const char *name;           // Имя устройства  
3     const char *phys;          // Физический путь  
4     const char *uniq;          // Уникальный идентифи  
        катор  
5  
6     unsigned long evbit[BITS_TO_LONGS(EV_CNT)]; // Поддерживаемые  
        типы событий  
7     unsigned long keybit[BITS_TO_LONGS(KEY_CNT)]; // Коды клавиш  
8     unsigned long relbit[BITS_TO_LONGS(REL_CNT)]; // Относительные  
        перемещения  
9     unsigned long absbit[BITS_TO_LONGS(ABS_CNT)]; // Абсолютные ко  
        ординаты  
10  
11     int (*open)(struct input_dev *dev); // Функция открытия  
12     void (*close)(struct input_dev *dev); // Функция закрытия
```

```

13
14     void *private;                // Приватные данные др
        айвера
15     struct device dev;            // Родительская структ
        ура device
16 };

```

1.4.2 Типы событий ввода

Основные типы событий, используемые в драйвере Xbox Series S:

Листинг 1.5 – Типы событий ввода

```

1 // Типы событий (evbit)
2 #define EV_KEY      0x01    // Клавишные события
3 #define EV_REL      0x02    // Относительные перемещения
4 #define EV_ABS      0x03    // Абсолютные координаты
5
6 // Коды кнопок мыши (keybit)
7 #define BTN_LEFT    0x110    // Левая кнопка мыши
8 #define BTN_RIGHT   0x111    // Правая кнопка мыши
9 #define BTN_MIDDLE   0x112    // Средняя кнопка мыши
10
11 // Относительные оси (relbit)
12 #define REL_X        0x00    // Перемещение по X
13 #define REL_Y        0x01    // Перемещение по Y

```

Драйвер Xbox Series S создаёт два устройства ввода:

1. Основное устройство геймпада со стандартными событиями контроллера;
2. Дополнительное устройство мыши для эмуляции функций мыши.

Для устройства мыши устанавливаются следующие типы событий:

- EV_KEY — для кнопок мыши (BTN_LEFT, BTN_RIGHT, BTN_MIDDLE);
- EV_REL — для относительного перемещения (REL_X, REL_Y, REL_WHEEL).

1.5 Протокол взаимодействия с Xbox Series S

Контроллер Xbox Series S использует протокол GIP (Game Input Protocol) с несколькими типами команд:

- GIP_CMD_INPUT (0x20) — основной пакет с данными о состоянии контроллера;
- GIP_CMD_POWER (0x05) — команды управления питанием;
- GIP_CMD_VIRTUAL_KEY (0x07) — виртуальные клавиши (кнопка Xbox).

1.5.1 Структура входного пакета

Листинг 1.6 – Структура пакета Xbox Series S

```
1 struct xboxone_input_report {
2     u8 command;                // Команда (0x20 для данных в в
    ода)
3     u8 reserved;              // Зарезервировано
4     u8 sequence;              // Последовательный номер
5     u8 length;                // Длина данных
6
7     // Байт 4: основные кнопки
8     // bit 0: Menu (Start)
9     // bit 1: View (Select)
10    // bit 2: A button
11    // bit 3: B button
12    // bit 4: X button
13    // bit 5: Y button
14    u8 buttons_main;
15
16    // Байт 5: дополнительные кнопки
17    u8 buttons_extra;
18
19    // Байты 6–9: триггеры (16–битные)
20    u16 left_trigger;
21    u16 right_trigger;
22
23    // Байты 10–13: левый стик (16–битные)
24    u16 left_stick_x;
```

```

25     u16 left_stick_y;
26
27     // Байты 14–17: правый стик (16–битные)
28     u16 right_stick_x;
29     u16 right_stick_y;
30 } __attribute__((packed));

```

Структура основного входного пакета включает:

- Байты 4-7: состояние кнопок (A, B, X, Y, START, SELECT и др.);
- Байты 10-17: значения аналоговых стиков (16-битные значения);
- Байты 6-9: значения триггеров.

Для инициализации контроллера требуется отправка специального пакета питания `xboxone_power_on`:

Листинг 1.7 – Пакет инициализации Xbox Series S

```

1 static const u8 xboxone_power_on[] = {
2     0x05,          // GIP_CMD_POWER
3     0x20,          // GIP_OPT_INTERNAL
4     0x00,          // GIP_SEQ0
5     0x01,          // Длина данных
6     0x00           // GIP_PWR_ON
7 };

```

1.6 Выводы

В результате проведённого анализа было определено следующее:

- Драйвер будет реализован в виде загружаемого модуля ядра Linux с использованием структур `usb_driver` и `module_usb_driver`;
- Для передачи данных будет использоваться структура `struct urb` с типом передачи `interrupt`;
- Будут созданы два устройства ввода (`struct input_dev`): геймпад и эмулятор мыши;
- Для эмуляции мыши будут преобразовываться данные аналоговых стиков в относительные движения курсора через события `EV_REL`;

- Кнопки А, В будут эмулировать левую и правую кнопки мыши соответственно через события EV_KEY;
- Правый стик будет использоваться для эмуляции прокрутки колесика мыши через события REL_WHEEL и REL_HWHEEL.

2 Конструкторский раздел

2.1 Последовательность действий ПО

На рисунке 2.1 представлена диаграмма IDEF0, описывающая последовательность действий ПО.

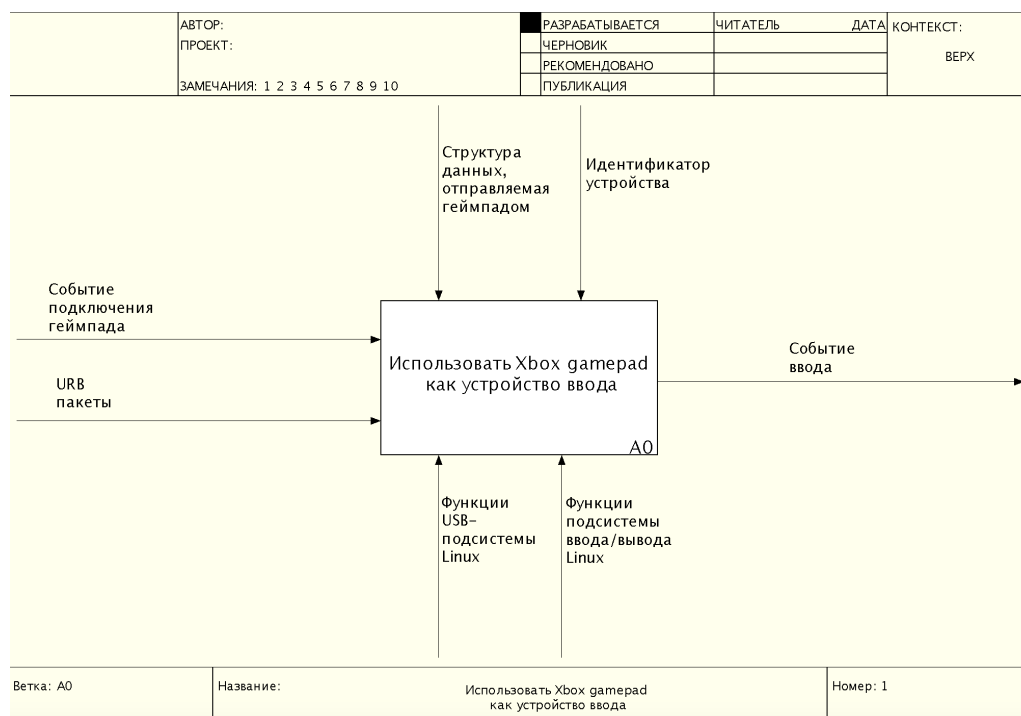


Рисунок 2.1 – Диаграмма IDEF0 нулевого уровня

2.2 Архитектура драйвера

Драйвер Xbox Series S имеет модульную архитектуру, состоящую из следующих основных компонентов:

- Модуль инициализации и регистрации USB-драйвера;
- Обработчик входящих пакетов данных от контроллера;
- Подсистема эмуляции мыши;
- Механизм отправки команд контроллеру.

На рисунке 2.2 представлена общая структура ПО.

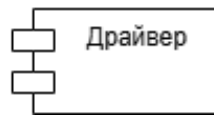


Рисунок 2.2 – Структура ПО

2.3 Алгоритм обработки ввода

На рисунке 2.3 представлена схема алгоритма обработки входящих данных от контроллера.

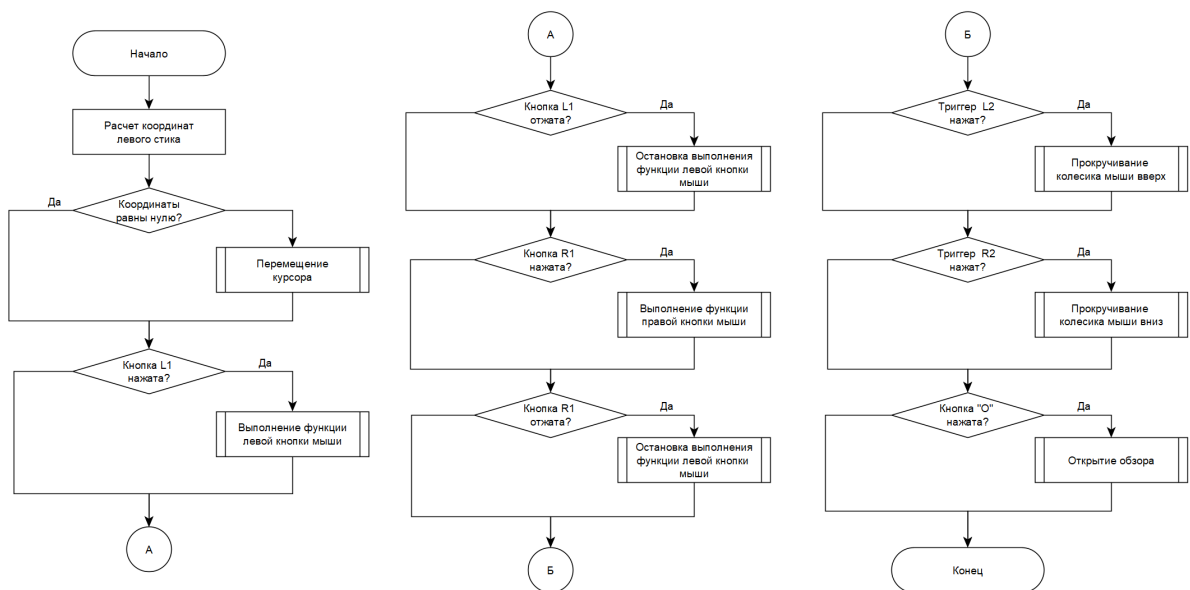


Рисунок 2.3 – Схема алгоритма обработки ввода

2.4 Алгоритм эмуляции мыши

Эмуляция мыши реализована через преобразование данных от аналоговых стиков:

- Левый стик (ось X) преобразуется в относительное горизонтальное перемещение курсора мыши;
- Правый стик (ось Y) преобразуется в относительное вертикальное перемещение курсора мыши;
- Кнопки контроллера эмулируют кнопки мыши: А → левая, В → правая, Х - средняя.

Для обработки перемещения курсора применяется алгоритм с мертвой зоной и регулируемой чувствительностью. На рисунке 2.4 представлена схема алгоритма перемещения курсора.



Рисунок 2.4 – Схема алгоритма перемещения курсора

Параметры настройки:

- `mouse_sensitivity` — чувствительность движения курсора (1-10);
- `scroll_sensitivity` — чувствительность прокрутки (1-10);
- `mouse_deadzone` — мертвая зона для исключения случайных движений.

2.5 Инициализация контроллера

Для корректной работы Xbox Series S контроллера требуется специальная последовательность инициализации:

1. Отправка пакета питания `xboxone_power_on`;
2. Ожидание подтверждения от контроллера;
3. Настройка конечных точек USB для приема и передачи данных;
4. Регистрация устройств ввода в системе.

3 Технологический раздел

3.1 Выбор языка и среды программирования

В качестве языка программирования был выбран язык **C** [**cdocs**], так как он является основным языком для разработки драйверов ядра Linux и обеспечивает прямой доступ к низкоуровневым функциям системы.

Для компиляции драйвера используется **gcc** [**gccdocs**] с флагами, соответствующими версии ядра Linux. В качестве среды разработки выбран текстовый редактор **Neovim** [**nvimdocs**] с плагинами для работы с кодом ядра.

Сборка драйвера осуществляется с помощью **Makefile**, который автоматизирует процесс компиляции и учитывает зависимости от версии ядра.

3.2 Функциональность разработанного ПО

Разработанный драйвер преобразует контроллер Xbox Series S в устройство ввода типа "мышь" с расширенными возможностями. Основная функциональность включает:

1. Назначение кнопок контроллера на функции мыши:
 - Кнопка A — левая кнопка мыши;
 - Кнопка B — правая кнопка мыши;
 - Кнопка X - средняя кнопка мыши.
2. Функцию перемещения курсора мыши с помощью стиков геймпада.
3. Регулируемые параметры чувствительности:
 - Чувствительность движения курсора;
 - Чувствительность прокрутки;
 - Размер мертвой зоны;
4. Автоматическое определение контроллера при подключении через USB.

Драйвер работает в фоновом режиме, автоматически определяя подключение контроллера и обеспечивая бесшовную интеграцию с существующей системой ввода Linux.

3.3 Структура данных драйвера

Основные структуры данных драйвера:

Листинг 3.1 – Структура драйвера Xbox Series S

```
1 struct usb_xpad {
2     struct input_dev *dev;           // Основное устройство ввода
3                                     // (геймпад)
4     struct usb_device *udev;         // USB-устройство
5     struct usb_interface *intf;      // USB-интерфейс
6
7     struct xpad_mouse *mouse;        // Структура для эмуляции мыши
8     unsigned char *idata;           // Буфер входящих данных
9     unsigned char *odata;           // Буфер исходящих данных
10
11     int mapping;                    // Карта преобразования ввода
12     int xtype;                     // Тип контроллера
13     const char *name;              // Имя устройства
14 };
15
16 struct xpad_mouse {
17     struct input_dev *dev;           // Устройство мыши
18     bool left_button;               // Состояние левой кнопки
19     bool right_button;              // Состояние правой кнопки
20     bool middle_button;             // Состояние средней кнопки
21     int deadzone;                   // Мертвая зона для стиков
22 };
```

3.4 Инициализация USB-драйвера

Листинг 3.2 – Инициализация USB-драйвера

```
1 static struct usb_device_id xpad_table[] = {
2     { USB_DEVICE(0x045e, 0x0b12) }, // Xbox Series S/X Controller
3     {}
4 };
5
6 static struct usb_driver xpad_driver = {
7     .name = "xpad_xbox_series",
8     .probe = xpad_probe,
9     .disconnect = xpad_disconnect,
10    .suspend = xpad_suspend,
11    .resume = xpad_resume,
```

```
12 |     .id_table = xpad_table ,  
13 | };
```

Поля структуры `usb_driver`:

- `name` — имя драйвера, отображаемое в системных логах;
- `probe` — функция, вызываемая при обнаружении поддерживаемого устройства;
- `disconnect` — функция, вызываемая при отключении устройства;
- `suspend/resume` — функции управления энергосбережением;
- `id_table` — таблица идентификаторов USB-устройств.

3.5 Параметры модуля

Драйвер поддерживает настройку через параметры модуля:

- `mouse_emulation` — включение/выключение эмуляции мыши;
- `mouse_sensitivity` — чувствительность движения курсора (1-10);
- `scroll_sensitivity` — чувствительность прокрутки (1-10).

3.6 Листинг загружаемого модуля ядра

Исходный код загружаемого модуля и файла сборки представлены в приложениях А и Б соответственно.

4 Исследовательский раздел

Для проверки работоспособности разработанного драйвера были проведены следующие тесты:

4.1 Загрузка модуля

На рисунке 4.1 показан лог загрузки модуля драйвера.

```
[10420.534920] xpad: Device Microsoft Xbox Series S|X Controller connected (VID:PID 045e:0b12)
[10420.534934] xpad: Mouse emulation enabled for this device
[10420.535326] input: Microsoft Xbox Series S|X Controller as /devices/pci0000:00/0000:00:14.0/usb1/1-2/1-2:1.0/input/input22
[10420.535879] input: Xbox Series S Mouse Emulation as /devices/pci0000:00/0000:00:14.0/usb1/1-2/1-2:1.0/input/input23
[10420.536312] xpad: Mouse emulation initialized for Microsoft Xbox Series S|X Controller
[10420.536320] xpad: Mouse sensitivity: 3, Scroll sensitivity: 2
[10420.536333] xpad: Mouse device mode enabled
```

Рисунок 4.1 – Загрузка модуля драйвера Xbox Series S

При загрузке модуля система выводит сообщение об инициализации драйвера и обнаружении устройства. Драйвер успешно определяет контроллер Xbox Series S, а также параметры.

4.2 Тестирование кнопок мыши

На рисунке 4.2 показан лог нажатий кнопок контроллера, преобразуемых в события мыши.

```
[10639.101117] xpad_mouse: Left button pressed
[10639.165080] xpad_mouse: Left button released
[10639.245075] xpad_mouse: Left button pressed
[10639.333084] xpad_mouse: Left button released
[10640.877152] xpad_mouse: Right button pressed
[10640.916952] xpad_mouse: Right button released
[10641.021024] xpad_mouse: Right button pressed
[10641.077149] xpad_mouse: Right button released
[10642.813148] xpad_mouse: Middle button pressed
[10642.941086] xpad_mouse: Middle button released
[10643.869150] xpad_mouse: Middle button pressed
[10643.988932] xpad_mouse: Middle button released
```

Рисунок 4.2 – Нажатие кнопок мыши (A/B/X)

4.3 Тестирование движения курсора

На рисунке 4.3 показан лог работы курсора при движении левого стика.

```
[10422.653065] xpad_mouse: Moved - X: -1, Y: 0 (raw: X=-11714, Y=291)
[10422.660945] xpad_mouse: Moved - X: -1, Y: 0 (raw: X=-12403, Y=291)
[10422.669068] xpad_mouse: Moved - X: -1, Y: 0 (raw: X=-12937, Y=291)
[10422.676951] xpad_mouse: Moved - X: -1, Y: 0 (raw: X=-13122, Y=291)
[10422.685096] xpad_mouse: Moved - X: -1, Y: 0 (raw: X=-12785, Y=291)
[10422.692966] xpad_mouse: Moved - X: -1, Y: 0 (raw: X=-11590, Y=291)
[10422.861063] xpad_mouse: Moved - X: -1, Y: 0 (raw: X=-12565, Y=291)
[10422.869151] xpad_mouse: Moved - X: -1, Y: 0 (raw: X=-13864, Y=291)
[10422.876937] xpad_mouse: Moved - X: -2, Y: 0 (raw: X=-25956, Y=291)
[10422.885077] xpad_mouse: Moved - X: -2, Y: 0 (raw: X=-26103, Y=291)
[10422.893134] xpad_mouse: Moved - X: -2, Y: 0 (raw: X=-26083, Y=291)
[10422.901129] xpad_mouse: Moved - X: -2, Y: 0 (raw: X=-25772, Y=291)
[10422.908951] xpad_mouse: Moved - X: -2, Y: 0 (raw: X=-25324, Y=291)
[10422.917128] xpad_mouse: Moved - X: -2, Y: 0 (raw: X=-24767, Y=291)
[10422.925193] xpad_mouse: Moved - X: -2, Y: 0 (raw: X=-24230, Y=291)
```

Рисунок 4.3 – Пример работы курсора

Движение стиков контроллера преобразуется в относительные движения курсора мыши. Реализована мертвая зона для исключения случайных движений и регулируемая чувствительность.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы были успешно решены поставленные задачи по разработке драйвера для геймпада Xbox Series S, выполняющего функции мыши.

Были выполнены следующие работы:

1. Проведён анализ USB-подсистемы ядра Linux и протоколов взаимодействия с контроллерами Xbox;
2. Исследована структура подсистемы ввода Linux и механизмы создания виртуальных устройств ввода;
3. Разработаны алгоритмы преобразования данных от аналоговых стиков в относительные движения курсора;
4. Реализован драйвер в виде загружаемого модуля ядра Linux с поддержкой параметрической настройки;
5. Проведено тестирование функций драйвера, подтвердившее его работоспособность.

В результате разработан драйвер, который:

- Автоматически определяет подключение контроллера Xbox Series S;
- Создаёт виртуальное устройство мыши в системе Linux;
- Выполняет функции кнопок и движения мыши;

ПРИЛОЖЕНИЕ А

```
1 #include <linux/version.h>
2 #include <linux/bits.h>
3 #include <linux/kernel.h>
4 #include <linux/input.h>
5 #include <linux/slab.h>
6 #include <linux/stat.h>
7 #include <linux/module.h>
8 #include <linux/usb/input.h>
9 #include <linux/timer.h>
10
11 #if LINUX_VERSION_CODE < KERNEL_VERSION(6,15,0)
12 #define timer_delete_sync del_timer_sync
13 #endif
14
15 #ifndef ABS_PROFILE
16 #define ABS_PROFILE ABS_MISC
17 #endif
18
19 #define XPAD_PKT_LEN 64
20
21 #define MAP_MOUSE_EMULATION      (1 << 0)
22
23 #define XTYPE_XBOXONE      3
24
25 static bool mouse_emulation = true;
26 module_param(mouse_emulation, bool, S_IRUGO);
27 MODULE_PARM_DESC(mouse_emulation, "Enable mouse emulation for
    Xbox Series S controller");
28
29 static int mouse_sensitivity = 3;
30 module_param(mouse_sensitivity, int, S_IRUGO);
31 MODULE_PARM_DESC(mouse_sensitivity, "Mouse sensitivity (1–10)");
32
33 static int scroll_sensitivity = 2;
34 module_param(scroll_sensitivity, int, S_IRUGO);
35 MODULE_PARM_DESC(scroll_sensitivity, "Scroll sensitivity (1–10)");
36
37 static bool mouse_debug = true;
38 module_param(mouse_debug, bool, S_IRUGO);
```

```

39 MODULE_PARM_DESC(mouse_debug, "Enable debug messages for mouse
    emulation");
40
41 static const struct xpad_device {
42     u16 idVendor;
43     u16 idProduct;
44     char *name;
45     u8 mapping;
46     u8 xtype;
47 } xpad_device[] = {
48     { 0x045e, 0x0b12, "Microsoft Xbox Series S|X Controller",
        MAP_MOUSE_EMULATION, XTYPE_XBOXONE },
49     { 0x0000, 0x0000, "Generic X-Box pad", 0, XTYPE_XBOXONE }
50 };
51
52 #define XPAD_XBOXONE_VENDOR_PROTOCOL(vend, pr) \
53     .match_flags = USB_DEVICE_ID_MATCH_VENDOR | \
54         USB_DEVICE_ID_MATCH_INT_INFO, \
55     .idVendor = (vend), \
56     .bInterfaceClass = USB_CLASS_VENDOR_SPEC, \
57     .bInterfaceSubClass = 71, \
58     .bInterfaceProtocol = (pr)
59 #define XPAD_XBOXONE_VENDOR(vend) \
60     { XPAD_XBOXONE_VENDOR_PROTOCOL((vend), 208) }
61
62 static const struct usb_device_id xpad_table[] = {
63     XPAD_XBOXONE_VENDOR(0x045e),
64     { }
65 };
66
67 MODULE_DEVICE_TABLE(usb, xpad_table);
68
69 #define GIP_CMD_POWER 0x05
70 #define GIP_CMD_VIRTUAL_KEY 0x07
71 #define GIP_CMD_INPUT 0x20
72
73 #define GIP_SEQ0 0x00
74 #define GIP_OPT_INTERNAL 0x20
75 #define GIP_PL_LEN(N) (N)
76 #define GIP_PWR_ON 0x00
77 #define GIP_WIRED_INTF_DATA 0

```



```

77
78 static const u8 xboxone_power_on[] = {
79     GIP_CMD_POWER, GIP_OPT_INTERNAL, GIP_SEQ0, GIP_PL_LEN(1),
80     GIP_PWR_ON
81 };
82 struct xpad_mouse {
83     struct input_dev *dev;
84     bool left_button;
85     bool right_button;
86     bool middle_button;
87     int deadzone;
88 };
89
90 struct usb_xpad {
91     struct usb_device *udev;
92     struct usb_interface *intf;
93
94     struct urb *irq_in;
95     unsigned char *idata;
96     dma_addr_t idata_dma;
97
98     struct urb *irq_out;
99     struct usb_anchor irq_out_anchor;
100    bool irq_out_active;
101    u8 odata_serial;
102    unsigned char *odata;
103    dma_addr_t odata_dma;
104    spinlock_t odata_lock;
105
106    char phys[64];
107    int mapping;
108    int xtype;
109    const char *name;
110
111    struct xpad_mouse *mouse;
112 };
113
114 static int xpad_init_mouse(struct usb_xpad *xpad)
115 {
116     struct input_dev *input_dev;

```

```

117     int error;
118
119     if (!(xpad->mapping & MAP_MOUSE_EMULATION))
120         return 0;
121
122     input_dev = input_allocate_device();
123     if (!input_dev)
124         return -ENOMEM;
125
126     xpad->mouse = kzalloc(sizeof(struct xpad_mouse), GFP_KERNEL);
127     if (!xpad->mouse) {
128         error = -ENOMEM;
129         goto err_free_input;
130     }
131
132     xpad->mouse->dev = input_dev;
133     xpad->mouse->deadzone = 8000;
134
135     input_dev->name = "Xbox Series S Mouse Emulation";
136     input_dev->phys = xpad->phys;
137     strlcat(input_dev->phys, "/mouse1", sizeof(input_dev->phys));
138     usb_to_input_id(xpad->udev, &input_dev->id);
139     input_dev->dev.parent = &xpad->intf->dev;
140
141     __set_bit(EV_KEY, input_dev->evbit);
142     __set_bit(EV_REL, input_dev->evbit);
143
144     __set_bit(BTN_LEFT, input_dev->keybit);
145     __set_bit(BTN_RIGHT, input_dev->keybit);
146     __set_bit(BTN_MIDDLE, input_dev->keybit);
147
148     __set_bit(REL_X, input_dev->relbit);
149     __set_bit(REL_Y, input_dev->relbit);
150     __set_bit(REL_WHEEL, input_dev->relbit);
151     __set_bit(REL_HWHEEL, input_dev->relbit);
152
153     error = input_register_device(input_dev);
154     if (error)
155         goto err_free_mouse;
156
157     printk(KERN_INFO "xpad: Mouse emulation initialized for

```

```

        %s\n", xpad->name);
158     printk(KERN_INFO "xpad: Mouse sensitivity: %d, Scroll
        sensitivity: %d\n",
159         mouse_sensitivity, scroll_sensitivity);
160     if (mouse_debug)
161         printk(KERN_INFO "xpad: Mouse debug mode enabled\n");
162
163     return 0;
164
165 err_free_mouse:
166     kfree(xpad->mouse);
167     xpad->mouse = NULL;
168 err_free_input:
169     input_free_device(input_dev);
170     return error;
171 }
172
173 static void xpad_deinit_mouse(struct usb_xpad *xpad)
174 {
175     if (xpad->mouse) {
176         if (xpad->mouse->dev) {
177             input_unregister_device(xpad->mouse->dev);
178             input_free_device(xpad->mouse->dev);
179         }
180         kfree(xpad->mouse);
181         xpad->mouse = NULL;
182     }
183 }
184
185 static void xpad_update_mouse(struct usb_xpad *xpad, unsigned
    char *data)
186 {
187     struct xpad_mouse *mouse = xpad->mouse;
188     if (!mouse) return;
189
190     int left_x_raw = (__s16)le16_to_cpup((__le16 *) (data + 10));
191     int left_y_raw = (__s16)le16_to_cpup((__le16 *) (data + 12));
192
193     int right_x_raw = (__s16)le16_to_cpup((__le16 *) (data + 14));
194     int right_y_raw = ~(__s16)le16_to_cpup((__le16 *) (data + 16));
195

```

```

196     int deadzone = mouse->deadzone;
197     int sens = clamp(mouse_sensitivity, 1, 10);
198     int scroll_sens = clamp(scroll_sensitivity, 1, 10);
199
200     int move_x = 0, move_y = 0;
201     int scroll_x = 0, scroll_y = 0;
202
203     if (abs(left_x_raw) > deadzone) {
204         move_x = (left_x_raw * sens) / 32767;
205         input_report_rel(mouse->dev, REL_X, move_x);
206     }
207     if (abs(left_y_raw) > deadzone) {
208         move_y = -(left_y_raw * sens) / 32767;
209         input_report_rel(mouse->dev, REL_Y, move_y);
210     }
211
212     if (abs(right_y_raw) > deadzone) {
213         scroll_y = -(right_y_raw * scroll_sens) / 32767;
214         input_report_rel(mouse->dev, REL_WHEEL, scroll_y);
215     }
216     if (abs(right_x_raw) > deadzone) {
217         scroll_x = (right_x_raw * scroll_sens) / 32767;
218         input_report_rel(mouse->dev, REL_HWHEEL, scroll_x);
219     }
220
221     if (mouse_debug && (move_x != 0 || move_y != 0 || scroll_x !=
222         0 || scroll_y != 0)) {
223         if (move_x != 0 || move_y != 0) {
224             printk(KERN_DEBUG "xpad_mouse: Moved — X: %d, Y: %d
225                 (raw: X=%d, Y=%d)\n",
226                 move_x, move_y, left_x_raw, left_y_raw);
227         }
228         if (scroll_x != 0 || scroll_y != 0) {
229             printk(KERN_DEBUG "xpad_mouse: Scrolled — Horizontal:
230                 %d, Vertical: %d (raw: X=%d, Y=%d)\n",
231                 scroll_x, scroll_y, right_x_raw, right_y_raw);
232         }
233     }
234
235     bool a_pressed = data[4] & BIT(4);
236     bool b_pressed = data[4] & BIT(5);

```

```

234     bool x_pressed = data[4] & BIT(6);
235
236     if (mouse_debug && (a_pressed != mouse->left_button ||
237                         b_pressed != mouse->right_button ||
238                         x_pressed != mouse->middle_button)) {
239         if (a_pressed != mouse->left_button) {
240             printk(KERN_DEBUG "xpad_mouse: Left button %s\n",
241                    a_pressed ? "pressed" : "released");
242         }
243         if (b_pressed != mouse->right_button) {
244             printk(KERN_DEBUG "xpad_mouse: Right button %s\n",
245                    b_pressed ? "pressed" : "released");
246         }
247         if (x_pressed != mouse->middle_button) {
248             printk(KERN_DEBUG "xpad_mouse: Middle button %s\n",
249                    x_pressed ? "pressed" : "released");
250         }
251     }
252
253     if (a_pressed != mouse->left_button) {
254         input_report_key(mouse->dev, BTN_LEFT, a_pressed);
255         mouse->left_button = a_pressed;
256     }
257
258     if (b_pressed != mouse->right_button) {
259         input_report_key(mouse->dev, BTN_RIGHT, b_pressed);
260         mouse->right_button = b_pressed;
261     }
262
263     if (x_pressed != mouse->middle_button) {
264         input_report_key(mouse->dev, BTN_MIDDLE, x_pressed);
265         mouse->middle_button = x_pressed;
266     }
267
268     input_sync(mouse->dev);
269 }
270
271 static void xpadone_process_packet(struct usb_xpad *xpad, u16
    cmd, unsigned char *data)
272 {
273     if ((xpad->mapping & MAP_MOUSE_EMULATION) && xpad->mouse &&

```

```

    data[0] == GIP_CMD_INPUT) {
274     xpad_update_mouse(xpad, data);
275 }
276 }
277
278 static void xpad_irq_in(struct urb *urb)
279 {
280     struct usb_xpad *xpad = urb->context;
281     struct device *dev = &xpad->intf->dev;
282     int retval, status;
283
284     status = urb->status;
285
286     switch (status) {
287     case 0:
288         break;
289     case -ECONNRESET:
290     case -ENOENT:
291     case -ESHUTDOWN:
292         dev_dbg(dev, "%s - urb shutting down with status: %d\n",
293             __func__, status);
294         return;
295     default:
296         dev_dbg(dev, "%s - nonzero urb status received: %d\n",
297             __func__, status);
298         goto exit;
299     }
300
301     xpadone_process_packet(xpad, 0, xpad->idata);
302
303 exit:
304     retval = usb_submit_urb(urb, GFP_ATOMIC);
305     if (retval)
306         dev_err(dev, "%s - usb_submit_urb failed with result
307             %d\n",
308             __func__, retval);
309 }
310
311 static void xpad_irq_out(struct urb *urb)
312 {
    struct usb_xpad *xpad = urb->context;

```

```

313     struct device *dev = &xpad->intf->dev;
314     int status = urb->status;
315
316     switch (status) {
317     case 0:
318         break;
319     case -ECONNRESET:
320     case -ENOENT:
321     case -ESHUTDOWN:
322         dev_dbg(dev, "%s - urb shutting down with status: %d\n",
323             __func__, status);
324         break;
325     default:
326         dev_dbg(dev, "%s - nonzero urb status received: %d\n",
327             __func__, status);
328         break;
329     }
330 }
331
332 static int xpad_init_output(struct usb_interface *intf, struct
    usb_xpad *xpad,
333     struct usb_endpoint_descriptor *ep_irq_out)
334 {
335     int error;
336
337     init_usb_anchor(&xpad->irq_out_anchor);
338
339     xpad->odata = usb_alloc_coherent(xpad->udev, XPAD_PKT_LEN,
340         GFP_KERNEL, &xpad->odata_dma);
341     if (!xpad->odata)
342         return -ENOMEM;
343
344     spin_lock_init(&xpad->odata_lock);
345
346     xpad->irq_out = usb_alloc_urb(0, GFP_KERNEL);
347     if (!xpad->irq_out) {
348         error = -ENOMEM;
349         goto err_free_coherent;
350     }
351
352     usb_fill_int_urb(xpad->irq_out, xpad->udev,

```

```

353         usb_sndintpipe(xpad->udev,
354             ep_irq_out->bEndpointAddress),
355         xpad->odata, XPAD_PKT_LEN,
356         xpad_irq_out, xpad, ep_irq_out->bInterval);
357 xpad->irq_out->transfer_dma = xpad->odata_dma;
358 xpad->irq_out->transfer_flags |= URB_NO_TRANSFER_DMA_MAP;
359
360     return 0;
361
362 err_free_coherent:
363     usb_free_coherent(xpad->udev, XPAD_PKT_LEN, xpad->odata,
364         xpad->odata_dma);
365     return error;
366 }
367
368 static void xpad_deinit_output(struct usb_xpad *xpad)
369 {
370     usb_free_urb(xpad->irq_out);
371     usb_free_coherent(xpad->udev, XPAD_PKT_LEN, xpad->odata,
372         xpad->odata_dma);
373 }
374
375 static int xpad_start_xbox_one(struct usb_xpad *xpad)
376 {
377     unsigned long flags;
378     int retval;
379
380     spin_lock_irqsave(&xpad->odata_lock, flags);
381
382     memcpy(xpad->odata, xboxone_power_on,
383         sizeof(xboxone_power_on));
384     xpad->odata[2] = xpad->odata_serial++;
385     xpad->irq_out->transfer_buffer_length =
386         sizeof(xboxone_power_on);
387
388     usb_anchor_urb(xpad->irq_out, &xpad->irq_out_anchor);
389     retval = usb_submit_urb(xpad->irq_out, GFP_ATOMIC);
390     if (retval) {
391         dev_err(&xpad->intf->dev,
392             "%s - usb_submit_urb failed with result %d\n",
393             __func__, retval);
394     }

```



```

389         usb_unanchor_urb(xpad->irq_out);
390     } else {
391         xpad->irq_out_active = true;
392     }
393
394     spin_unlock_irqrestore(&xpad->odata_lock, flags);
395
396     return retval;
397 }
398
399 static int xpad_start_input(struct usb_xpad *xpad)
400 {
401     int error;
402
403     if (usb_submit_urb(xpad->irq_in, GFP_KERNEL))
404         return -EIO;
405
406     error = xpad_start_xbox_one(xpad);
407     if (error) {
408         usb_kill_urb(xpad->irq_in);
409         return error;
410     }
411
412     return 0;
413 }
414
415 static void xpad_stop_input(struct usb_xpad *xpad)
416 {
417     usb_kill_urb(xpad->irq_in);
418 }
419
420 static void xpad_deinit(struct usb_xpad *xpad)
421 {
422     xpad_deinit_mouse(xpad);
423 }
424
425 static int xpad_probe(struct usb_interface *intf, const struct
usb_device_id *id)
426 {
427     struct usb_device *udev = interface_to_usbdev(intf);
428     struct usb_xpad *xpad;

```

```

429     struct usb_endpoint_descriptor *ep_irq_in, *ep_irq_out;
430     int i, error;
431
432     if (intf->cur_altsetting->desc.bNumEndpoints != 2)
433         return -ENODEV;
434
435     for (i = 0; xpad_device[i].idVendor; i++) {
436         if ((le16_to_cpu(udev->descriptor.idVendor) ==
437             xpad_device[i].idVendor) &&
438             (le16_to_cpu(udev->descriptor.idProduct) ==
439             xpad_device[i].idProduct))
440             break;
441     }
442
443     xpad = kzalloc(sizeof(*xpad), GFP_KERNEL);
444     if (!xpad)
445         return -ENOMEM;
446
447     usb_make_path(udev, xpad->phys, sizeof(xpad->phys));
448     strlcat(xpad->phys, "/input0", sizeof(xpad->phys));
449
450     xpad->idata = usb_alloc_coherent(udev, XPAD_PKT_LEN,
451                                     GFP_KERNEL, &xpad->idata_dma);
452     if (!xpad->idata) {
453         error = -ENOMEM;
454         goto err_free_mem;
455     }
456
457     xpad->irq_in = usb_alloc_urb(0, GFP_KERNEL);
458     if (!xpad->irq_in) {
459         error = -ENOMEM;
460         goto err_free_idata;
461     }
462
463     xpad->udev = udev;
464     xpad->intf = intf;
465     xpad->mapping = xpad_device[i].mapping;
466     xpad->xtype = xpad_device[i].xtype;
467     xpad->name = xpad_device[i].name;
468
469     if (mouse_emulation)

```

```

468         xpad->mapping |= MAP_MOUSE_EMULATION;
469
470     printk(KERN_INFO "xpad: Device %s connected (VID:PID
        %04x:%04x)\n",
471           xpad->name, le16_to_cpu(udev->descriptor.idVendor),
472           le16_to_cpu(udev->descriptor.idProduct));
473
474     if (xpad->mapping & MAP_MOUSE_EMULATION) {
475         printk(KERN_INFO "xpad: Mouse emulation enabled for this
            device\n");
476     }
477
478     if (xpad->xtype == XTYPE_XBOXONE &&
479         intf->cur_altsetting->desc.bInterfaceNumber !=
            GIP_WIRED_INTF_DATA) {
480         error = -ENODEV;
481         goto err_free_in_urb;
482     }
483
484     ep_irq_in = ep_irq_out = NULL;
485
486     for (i = 0; i < 2; i++) {
487         struct usb_endpoint_descriptor *ep =
488             &intf->cur_altsetting->endpoint[i].desc;
489
490         if (usb_endpoint_xfer_int(ep)) {
491             if (usb_endpoint_dir_in(ep))
492                 ep_irq_in = ep;
493             else
494                 ep_irq_out = ep;
495         }
496     }
497
498     if (!ep_irq_in || !ep_irq_out) {
499         error = -ENODEV;
500         goto err_free_in_urb;
501     }
502
503     error = xpad_init_output(intf, xpad, ep_irq_out);
504     if (error)
505         goto err_free_in_urb;

```

```

506
507     usb_fill_int_urb(xpad->irq_in, udev,
508                     usb_rcvintpipe(udev, ep_irq_in->bEndpointAddress),
509                     xpad->idata, XPAD_PKT_LEN, xpad_irq_in,
510                     xpad, ep_irq_in->bInterval);
511     xpad->irq_in->transfer_dma = xpad->idata_dma;
512     xpad->irq_in->transfer_flags |= URB_NO_TRANSFER_DMA_MAP;
513
514     usb_set_intfdata(intf, xpad);
515
516     error = xpad_init_mouse(xpad);
517     if (error) {
518         dev_warn(&xpad->intf->dev,
519                 "unable to initialize mouse emulation: %d\n", error);
520         goto err_deinit_output;
521     }
522
523     error = xpad_start_input(xpad);
524     if (error) {
525         dev_err(&xpad->intf->dev,
526                 "unable to start input: %d\n", error);
527         goto err_deinit_all;
528     }
529
530     return 0;
531
532 err_deinit_all:
533     xpad_deinit(xpad);
534 err_deinit_output:
535     xpad_deinit_output(xpad);
536 err_free_in_urb:
537     usb_free_urb(xpad->irq_in);
538 err_free_idata:
539     usb_free_coherent(udev, XPAD_PKT_LEN, xpad->idata,
540                       xpad->idata_dma);
540 err_free_mem:
541     kfree(xpad);
542     return error;
543 }
544
545 static void xpad_disconnect(struct usb_interface *intf)

```

```

546 {
547     struct usb_xpad *xpad = usb_get_intfdata(intf);
548
549     printk(KERN_INFO "xpad: Device %s disconnected\n",
550             xpad->name);
551
552     xpad_stop_input(xpad);
553     xpad_deinit(xpad);
554     xpad_deinit_output(xpad);
555     usb_free_urb(xpad->irq_in);
556     usb_free_coherent(xpad->udev, XPAD_PKT_LEN, xpad->idata,
557                       xpad->idata_dma);
558     kfree(xpad);
559
560     usb_set_intfdata(intf, NULL);
561 }
562
563 static struct usb_driver xpad_driver = {
564     .name          = "xpad",
565     .probe         = xpad_probe,
566     .disconnect    = xpad_disconnect,
567     .id_table      = xpad_table,
568 };
569
570 module_usb_driver(xpad_driver);
571
572 MODULE_AUTHOR("Trokhan Andrey");
573 MODULE_DESCRIPTION("Xbox gamepad mouse emulation driver");
574 MODULE_LICENSE("GPL");

```

ПРИЛОЖЕНИЕ Б

```
1 obj-m = xpad.o
2
3 KVERSION = $(shell uname -r)
4 all:
5     make -C /lib/modules/$(KVERSION)/build M=$(PWD) modules
6
7 clean:
8     make -C /lib/modules/$(KVERSION)/build M=$(PWD) clean
```